

技术报告

长上下文场景中大模型自动数据标注方案

队名	Agent los
成员	金冲
学校	河北工业大学
联系方式	3408201667@qq.com
提交材料	技术报告、预测结果 jsonl 压缩包、可复现代码压缩包
报告日期	2026 年 5 月

摘要

本方案面向 FlagOS / OpenSeek 赛道三“长上下文场景中大模型自动数据标注”任务，在官方约束下使用 Qwen3-4B 作为基础模型，围绕 8 个测试集生成统一格式的 jsonl 预测结果。方案的核心目标不是单纯堆叠更长输入，而是在有限上下文窗口中构造信息密度更高、格式更稳定、可自动恢复的标注流程。

整体技术路线由四个部分组成：第一，构建统一的长上下文 ICL 提示模板，将角色定义、任务说明、示例区域、待标注文本和输出格式约束固定化；第二，使用 tokenizer 估计示例 token 长度，在上下文预算内选择尽可能多且格式一致的官方示例；第三，对不同任务采用专用提示与后处理策略，包括数值计算、词性计数、字符串拼接、情感分类、体裁判断、Jeopardy 问答和 Triton Kernel 代码生成；第四，建立并发推理、断点续传、重试补漏和结果校验流程，保证大批量自动标注的可扩展性与一致性。

终提交文件包含 8 个 jsonl 结果文件，均通过本地 JSON 解析、字段完整性与非空预测检查。其中任务 1 至任务 7 各包含 500 条预测，任务 8 包含 166 条预测，统一打包为 submission_Jin_Final_Strict.zip。

一、赛题理解与约束

赛题要求参赛团队在超长上下文环境中使用大模型完成自动数据标注。与常规分类或生成任务相比，本赛题有三个关键难点：一是可用标注示例数量多，直接拼接会超过模型上下文容量；二是不同任务的输出形态差异大，既有短标签、数值、字符串，也有较长代码；三是批量推理过程中容易出现标签缺失、思考过程泄露、格式漂移、请求超时和局部样本失败等工程问题。

本方案遵守以下约束：仅使用官方提供的数据与示例，不引入外部标注数据；模型侧使用 Qwen34B；底层模型加载与运行面向官方 Ascend / FlagScale 环境，推理脚本不直接耦合底层设备细节，而是通过本地 OpenAI-compatible Chat Completions 接口访问模型服务，服务地址为 `http://localhost:8000/v1/chat/completions`；所有提交结果均为 jsonl，每行包含 `test_sample_id` 与 `prediction` 字段。

任务覆盖情况

任务	数据集	输出类型	本地结果条数	主要策略
1	closest integers	数字/短答案	500	通用 ICL 模板、标签抽取
2	count nouns / verbs	整数	500	词性计数专用提示、数字兜底抽取
3	Collatz conjecture	整数或整数列表	500	单步规则拆解、逐数处理、三层数字抽取
4	CoNaLa concat strings	字符串	500	拼接专用提示、Python 语法兜底
5	tweet sadness detection	Sad / Not sad	500	情感分类专用提示、标签标准化
6	MNLI same genre	Y / N	500	体裁定义注入、二分类格式约束
7	Jeopardy answer generation	实体短语	500	短答案提示、小写规范化、unknown 补漏
8	kernel generation	Python/Triton 代码	166	代码块约束、长输出 token 预算、缺失样本补齐

二、系统架构

系统以“模型服务层、提示构造层、任务推理层、输出治理层”四层组织。模型服务层负责加载 Qwen3-4B 并对外提供本地接口；提示构造层根据任务说明、官方示例和待标注样本生成上下文 prompt；任务推理层负责并发请求、超时控制和断点续传；输出治理层负责从模型回复中提取 终答案、清洗格式并写入 jsonl。

层级	功能	对应代码
模型服务层	启动 Qwen3-4B，本地暴露 Chat Completions API；推理脚本统一访问 8000 端口。	code/api_test.py、code/method.py
提示构造层	生成角色、任务、示例、待标注文本、 终输出要求的结构化 prompt。	build_prompt、select_examples
任务推理层	读取 8 个任务数据，按样本并发请求模型，支持已完成 ID 跳过。	code/main.py、code/debug_*.py
输出治理层	删除 <think> 片段，抽取 <label>，进行任务级兜底和标准化。	count_answer、各任务 request_* 函数

在工程实现中，通用入口 main.py 提供了全任务可复用的流程：读取任务 JSON，获取任务定义、示例与测试样本，构造 prompt，并通过线程池并发推理。针对格式更敏感或规律更明确的任务，则使用 debug_two.py 至 debug_eight2.py 等专用脚本进行增强，这些脚本保留相同的提交输出格式，便于终合并与打包。

三、模型指令与提示策略

3.1 统一提示模板

通用模板由五个固定区域组成：角色定义、核心任务、关键标注准则、示例区域、待标注文本。角色定义将模型限定为专业数据标注员，减少闲聊式回答；核心任务直接插入官方任务定义，保持与数据集目标一致；关键标注准则反复强调输出必须使用 <label>...</label> 包裹；示例区域承载官方 ICL 样例；待标注文本位于 prompt 末尾，以降低被大量示例淹没的风险。

```
### Role Definition
You are a professional data annotation expert specialized in long-context text labeling.

### Core Task
{task_description}

### Examples (Must Be Fully Followed)
[[EXAMPLES]]

### Text to Annotate
{text2annotate}

### Final Requirement Summary
The final annotation result MUST be wrapped in <label> tags.
```

该结构的优点是稳定、可复用且便于替换示例。对于模型容易输出解释的情况，后处理阶段会删除 `<think>` 到 `</think>` 的内容，再提取 后一个有效标签，优先相信靠近回答末尾的“ 终结论”。

3.2 任务专用提示

通用 ICL 模板可以覆盖多数短标签任务，但对数值计算、字符串拼接和代码生成等任务，单纯“学习示例”并不总是稳定。因此本方案为任务 2 至任务 8 设计了更窄的指令，把输出空间压缩到 小。

- 词性计数任务要求只输出单个整数，并明确禁止 “Nouns: X, Verbs: Y” 一类冗余格式。
- Collatz 任务强调只做一步计算，避免模型继续推导完整序列。
- 字符串拼接任务要求不添加空格，同时使用本地 `ast.literal_eval` 和 `"".join()` 作为可靠兜底。
- 情感分类任务将输出限定为 `Sad` 或 `Not sad`，并在解析阶段统一大小写与同义表达。
- MNLI 体裁判断任务把官方体裁定义写入 `prompt`，降低模型对 `genre` 名称的误解。
- Jeopardy 任务要求答案实体简短且全小写，后处理阶段强制 `lower()`。
- Kernel 生成任务要求只输出 Python/Triton 代码块，减少解释文字对评测脚本的干扰。

四、超长上下文输入构造

当可用示例数量显著超过模型上下文容量时，关键问题不是“尽量塞满”，而是“在预算内保留高信号内容”。本方案采用 token 预算驱动的示例组织方式：先保留任务说明和待标注文本所需空间，再使用 Qwen3-4B tokenizer 估计示例输入与输出长度，逐条加入官方示例，直到达到目标长度阈值。

当前代码中 `select_examples` 使用 `AutoTokenizer.from_pretrained` 加载 Qwen3-4B tokenizer，并将目标示例区长度设为 8192 token。每条示例被统一格式化为 `# input <label> output </label>`。这样做有三点收益：其一，示例格式与 终答案格式一致；其二，token 预算可控，降低超长 prompt 被截断的风险；其三，所有样本采用相同构造逻辑，便于复现。

```
for example in all_examples:
    input_tokens = len(tokenizer.encode(input_text, add_special_tokens=False))
    output_tokens = len(tokenizer.encode(output_text, add_special_tokens=False))    if
    token_num + input_tokens + output_tokens + format_tokens <= target_length:
        examples_str += f"# {input_text} <label> {output_text} </label>\n"
```

在信息密度方面，任务 1 至任务 7 更依赖短示例与稳定格式；任务 8 的待生成内容更长，示例堆叠对终代码质量的收益有限，因此更重视“明确代码输出协议”和“足够 completion token”。后续开源版本可进一步加入 BM25 或 TF-IDF 检索，将“按顺序加入示例”升级为“按相似度与标签平衡选择示例”，但终提交版优先选择了低复杂度、易复现、稳定性更高的上下文构造。

五、自动多轮与持续交互中的一致性

本方案没有把所有样本放入同一个长会话中连续追问，而是采用“样本级独立推理 + 自动重试补漏”的持续交互方式。这样可以避免早期错误污染后续上下文，也便于失败样本单独重跑。持续交互的一致性由以下机制保证：

1. 固定输入协议：每次请求都使用相同的任务模板、示例格式和输出约束。
2. 固定采样参数：确定性任务通常使用 `temperature=0.0`，补漏任务仅在必要时使用很低温度。
3. 断点续传：每次运行先读取已生成 jsonl，建立 `processed_ids` 集合，跳过已完成样本。
4. 失败不占坑：通用流程中无法抽取有效 prediction 时不写入结果文件，便于后续自动补跑。
5. 补漏脚本：对 `unknown`、缺失 ID 或代码生成失败样本使用单线程、更长超时、更强约束 prompt 重试。

这种设计将“多轮对话”转化为可审计的批处理状态机：每个样本的状态只由是否已写入合法 jsonl 决定，任务脚本可以反复运行，直到所有样本都有非空 prediction。相比把模型会话历史持续增长，该方式更可扩展，也更适合在每天提交次数有限的竞赛环境中稳定迭代。

六、输出解析与质量控制

Qwen3-4B 在推理时可能输出思考过程、解释文字、半截标签或多次候选答案。为避免格式漂移影响提交，本方案在输出端设置了多层防线。

问题	处理方式	收益
输出包含 <think>	正则删除思考片段后再解析。	避免内部推理干扰 终标签提取。
存在多个 <label>	优先选择 后一个匹配结果。	贴近模型“后给 终答案”的行为。
标签缺失但答案明显	从清洗后文本末尾提取数字、Y/N、Sad 等合法值。	降低格式错误导致的空结果。
请求超时或接口异常	设置 retry 与 timeout，失败样本不写入或后续补跑。	保证批处理不会因单条样本中断。
代码生成缺失	使用代码块抓取、去除 markdown 包裹，必要时写入语法合法兜底代码。	保证提交文件结构合法。

终提交前，本地对 8 个结果文件进行了完整性检查：逐行解析 JSON，检查 test_sample_id 非空，检查 prediction 非空。检查结果显示 8 个文件均无 JSON 解析错误、无空预测。

七、实验与提交物状态

由于赛事平台榜单按历史 佳成绩排名，实验迭代遵循“先闭环、再优化、后锁版本”的原则。第一阶段打通本地模型服务与单条 API；第二阶段完成 8 个任务的全量输出；第三阶段针对高风险任务进行专用 prompt 和补漏；后将 8 个 jsonl 与代码打包为 终提交文件。

文件	行数	JSON 错误	空 prediction	状态
openseek-1-v1.jsonl	500	0	0	可提交
openseek-2-v1.jsonl	500	0	0	可提交
openseek-3-v1.jsonl	500	0	0	可提交
openseek-4-v1.jsonl	500	0	0	可提交
openseek-5-v1.jsonl	500	0	0	可提交
openseek-6-v1.jsonl	500	0	0	可提交
openseek-7-v1.jsonl	500	0	0	可提交
openseek-8-v1.jsonl	166	0	0	可提交

终压缩包 submission_Jin_Final_Strict.zip 包含上述 8 个预测文件，并同时保留推理代码目录 code/。其中 main.py 是通用推理入口，method.py 包含 prompt 构造、示例选择、模型请求和答案解析，debug_*.py 是任务专用增强脚本。

八、可复现说明

8.1 环境依赖

核心 Python 依赖包括 requests、tqdm、transformers。模型服务侧需在官方 Ascend 910C 与 FlagScale 运行环境中部署 Qwen3-4B，并确保推理服务兼容 OpenAI Chat Completions 请求格式。代码层将模型服务视为本地推理后端，因此同一套推理脚本可以在服务重启后继续断点续传。

```
pip install requests tqdm transformers
```

建议在代码包中提供 requirements.txt，并在 README 中写明模型权重路径、服务端口、数据目录与输出目录。推理脚本默认假设模型名为 Qwen3-4B，服务地址为 http://127.0.0.1:8000/v1/chat/completions。

8.2 服务连通性检查

```
cd code
python api_test.py
```

该脚本会向本地 8000 端口发送一次 Chat Completions 请求，并打印返回内容与 token 用量，用于确认模型服务是否可用。

8.3 通用推理命令

```
python main.py --task_id 1 --max_input_length 32000 --tokenizer_path  
/root/OpenSeek/openseek/competition/Qwen3-4B --workers 4 python  
main.py --task_id 2 --max_input_length 32000 --tokenizer_path  
/root/OpenSeek/openseek/competition/Qwen3-4B --workers 4
```

对于任务 2 至任务 8，建议优先使用专用脚本生成 终结果，例如 `python debug_two.py`、`python debug_six.py`、`python debug_eight.py`。专用脚本均带有断点续传逻辑，重复运行时会跳过已完成样本。

8.4 结果打包

```
zip submission_Jin_Final_Strict.zip openseek-1-v1.jsonl openseek-2-v1.jsonl openseek-3v1.jsonl  
openseek-4-v1.jsonl openseek-5-v1.jsonl openseek-6-v1.jsonl openseek-7-v1.jsonl openseek-8-  
v1.jsonl
```

提交前应至少检查文件数量、JSON 合法性、字段完整性、空预测比例和压缩包内路径。平台每天多提交 5 次，因此本地校验是避免浪费提交次数的关键。

九、风险分析与改进方向

当前方案以稳定提交为优先，因此在部分任务上采用了较强的规则化提示和后处理。其主要风险包括：模型对复杂代码生成任务的正确性不足；部分数值或知识型题目可能受模型能力上限影响；示例选择目前偏向确定性与可复现，尚未充分利用相似度检索带来的潜在收益。

后续优化可以从三个方向展开。第一，引入 BM25 / TF-IDF / embedding 召回，在官方示例集中为每条测试样本动态选择更相似的 Top-K 示例，并保持标签类别平衡。第二，建立任务级小规模验证集，对 prompt 版本、示例数量、排序策略、temperature 和 max tokens 做消融实验。第三，对 Kernel 生成任务引入语法检查与 小运行测试，优先筛掉明显无法执行的 Triton 代码。

十、开源计划

根据赛事要求，2026 年 5 月 21 日至 2026 年 5 月 31 日期间，将把技术报告与可复现代码提交到 GitHub OpenSeek 官方开源项目，并在 FlagOS 赛事平台回填 PR 链接。开源内容计划包含：完整推

- 理代码：通用入口、任务专用脚本、API 测试脚本与结果校验脚本。
- 环境文件：requirements.txt 与必要的服务启动说明。
- README：数据目录约定、模型服务启动、8 个任务推理命令、结果打包命令。
- 技术报告 PDF：即本报告的 终提交版本。

结论

本方案围绕长上下文数据标注的核心问题，构建了可复现、可恢复、可扩展的自动标注流程。提示层通过结构化 ICL 与任务专用约束提高模型输出稳定性；上下文层通过 tokenizer 预算控制示例规模，避免无效超长输入；工程层通过并发推理、断点续传、重试补漏和本地校验保证批量结果完整。 终提交物覆盖 8 个测试集，满足 jsonl 格式和压缩包提交要求，并为后续 OpenSeek 开源 PR 保留了清晰的代码与文档结构。

附：本地文件检查摘要

submission_Jin_Final_Strict.zip 已包含 8 个预测结果文件与 code/ 目录；8 个 jsonl 文件均可逐行解析，且 test_sample_id 与 prediction 字段非空。